

Relational Database

Core Concepts & Syntax Guide

Tables · Rows · Columns · Keys · Schema · Index · Constraints · Relationships · ACID
Complete Syntax · Parameters · Examples · Reference Tables

Detailed Study Guide with Full SQL Syntax

Relational Databases — Complete Reference

Table of Contents

- 1. Table (Relation)** — CREATE TABLE · ALTER TABLE · DROP TABLE · TRUNCATE
- 2. Row / Tuple** — INSERT INTO · UPDATE · DELETE · SELECT single row
- 3. Column / Attribute** — Column definition syntax · data types · modifying columns
- 4. Primary Key** — PRIMARY KEY syntax · composite · surrogate · ALTER ADD PK
- 5. Foreign Key** — FOREIGN KEY · REFERENCES · ON DELETE/UPDATE actions
- 6. Schema** — CREATE / DROP / USE DATABASE · schema design syntax
- 7. Index** — CREATE INDEX · types · DROP INDEX · EXPLAIN
- 8. Constraints** — NOT NULL · UNIQUE · CHECK · DEFAULT · named constraints
- 9. Relationships** — JOIN syntax · 1:1 · 1:N · M:N junction table queries
- 10. ACID Properties** — BEGIN · COMMIT · ROLLBACK · SAVEPOINT · isolation levels

1

Table (Relation)

The fundamental 2-D storage structure of a relational database

A **table** stores data for a single entity as rows and columns. Every column has a fixed name, data type, and optional constraints. Tables are created with **CREATE TABLE**, modified with **ALTER TABLE**, and removed with **DROP TABLE**.

CREATE TABLE — Full Syntax

SYNTAX — CREATE TABLE

```
CREATE TABLE [IF NOT EXISTS] TABLE_name (  
    COLUMN1 datatype [COLUMN_CONSTRAINTS],  
    COLUMN2 datatype [COLUMN_CONSTRAINTS],  
    ...  
    [TABLE_CONSTRAINTS]  
);
```

Note: IF NOT EXISTS prevents error if table already exists

Column constraint options (after the data type):

Constraint	Syntax	Meaning
Primary Key	PRIMARY KEY	This column uniquely identifies each row
Not Null	NOT NULL	Column must always have a value
Unique	UNIQUE	All values in column must be distinct
Default	DEFAULT value	Value used when INSERT omits this column
Check	CHECK(condition)	Value must satisfy the condition
Auto Increment	AUTO_INCREMENT (MySQL) / SERIAL (PostgreSQL)	Assigns sequential integers
Foreign Key	REFERENCES table(col)	Links to another table's primary key

CREATE TABLE — Complete Real Example

```
CREATE TABLE Students (  
    StudentID INT PRIMARY KEY AUTO_INCREMENT, FirstName VARCHAR(50)  
    NOT NULL, LastName VARCHAR(50) NOT NULL, Email VARCHAR(100) UNIQUE NOT NULL, Age INT  
    CHECK(Age BETWEEN 16 AND 60), DeptID INT DEFAULT 1, CreatedAt DATETIME DEFAULT  
    CURRENT_TIMESTAMP, FOREIGN KEY (DeptID) REFERENCES Departments(DeptID) );
```

ALTER TABLE — Full Syntax

SYNTAX — ALTER TABLE

```
-- ADD a new COLUMN
ALTER TABLE TABLE_name ADD [COLUMN] col_name datatype [CONSTRAINTS];

-- Modify an existing COLUMN's type or CONSTRAINTS
ALTER TABLE TABLE_name MODIFY [COLUMN] col_name new_datatype [CONSTRAINTS];

-- Rename a COLUMN
ALTER TABLE TABLE_name RENAME COLUMN old_name TO new_name;

-- DROP a COLUMN
ALTER TABLE TABLE_name DROP [COLUMN] col_name;

-- ADD a CONSTRAINT
ALTER TABLE TABLE_name ADD CONSTRAINT name PRIMARY KEY (col);

-- DROP a CONSTRAINT
ALTER TABLE TABLE_name DROP CONSTRAINT CONSTRAINT_name;
```

Note: MODIFY is MySQL syntax; PostgreSQL uses ALTER COLUMN

Examples:

```
ALTER TABLE Students ADD COLUMN Phone VARCHAR(15); -- add phone
ALTER TABLE Students MODIFY COLUMN Age TINYINT NOT NULL; -- change type
ALTER TABLE Students DROP COLUMN Phone; -- remove phone
ALTER TABLE Students RENAME COLUMN LastName TO Surname; -- rename
```

DROP TABLE & TRUNCATE TABLE

SYNTAX — DROP & TRUNCATE

```
-- Permanently DELETE the TABLE AND all its data
DROP TABLE [IF EXISTS] TABLE_name;

-- Remove ALL rows but keep the TABLE structure
TRUNCATE TABLE TABLE_name;

-- DROP multiple TABLEs at ONce
DROP TABLE IF EXISTS TABLE1, TABLE2, TABLE3;
```

Note: DROP cannot be rolled back in most databases

Command	Removes Data	Removes Structure	Can ROLLBACK	Resets AUTO_INCREMENT
DELETE	Yes (row-by-row)	No	Yes	No
TRUNCATE	Yes (all rows)	No	No (usually)	Yes
DROP	Yes (all rows)	Yes	No	N/A

A **row** represents one complete instance of the table's entity. Rows are added with **INSERT**, changed with **UPDATE**, removed with **DELETE**, and read with **SELECT**.

INSERT INTO — Full Syntax

SYNTAX — INSERT INTO

```
-- INSERT One row (specify COLUMNS explicitly – recommended)
INSERT INTO TABLE_name (col1, col2, col3, ...)
VALUES (val1, val2, val3, ...);

-- INSERT WITHout COLUMN list (must match ALL COLUMNS in order)
INSERT INTO TABLE_name VALUES (val1, val2, ...);

-- INSERT multiple rows in ONE statement
INSERT INTO TABLE_name (col1, col2)
VALUES (v1a, v2a),
      (v1b, v2b),
      (v1c, v2c);

-- INSERT FROM another TABLE
INSERT INTO archive_TABLE (col1, col2)
SELECT col1, col2 FROM source_TABLE WHERE cONdition;
```

Examples:

```
-- Single row INSERT INTO Students (FirstName, LastName, Email, Age, DeptID) VALUES
('Arjun', 'Sharma', 'arjun@college.edu', 20, 1); -- Multiple rows at once INSERT INTO
Students (FirstName, LastName, Email, Age, DeptID) VALUES ('Priya', 'Nair',
'priya@college.edu', 19, 2), ('Rahul', 'Verma', 'rahul@college.edu', 21, 1), ('Sneha',
'Rao', 'sneha@college.edu', 20, 3); -- Copy rows from another table INSERT INTO Alumni
(StudentID, Name) SELECT StudentID, FirstName FROM Students WHERE GraduationYear = 2024;
```

UPDATE — Full Syntax

SYNTAX — UPDATE

```
UPDATE TABLE_name
SET col1 = value1,
    col2 = value2,
    ...
WHERE cONdition;

-- UPDATE using value FROM another TABLE (JOIN)
UPDATE TABLE1 t1
JOIN  TABLE2 t2 ON t1.id = t2.id
SET   t1.col = t2.col
WHERE t2.cONdition;
```

Note: Always use WHERE — omitting it updates EVERY row in the table

```
UPDATE Students SET Age = 21 WHERE StudentID = 1; -- one row
UPDATE Students SET DeptID = 2 WHERE LastName = 'Sharma'; -- multiple rows
UPDATE Students SET Age = Age + 1; -- ALL rows (no WHERE!)
UPDATE Students SET Email = LOWER(Email) WHERE Email != LOWER(Email);
```

DELETE — Full Syntax

SYNTAX — DELETE

```
-- DELETE specific rows
DELETE FROM TABLE_name WHERE cONdition;

-- DELETE all rows (logged, can ROLLBACK, slower than TRUNCATE)
DELETE FROM TABLE_name;

-- DELETE WITH JOIN (MySQL)
DELETE t1 FROM TABLE1 t1
JOIN  TABLE2 t2 ON t1.id = t2.fk_id
WHERE t2.status = 'inactive';
```

Note: Always test your WHERE with SELECT before running DELETE

```
DELETE FROM Students WHERE StudentID = 5; -- remove one student
DELETE FROM Students WHERE DeptID = 3 AND Age < 18; -- conditional
DELETE FROM Enrollments WHERE CourseID = 'CS101'; -- delete related rows
```

A **column** defines the name, type, and rules for one piece of data across all rows. Columns are declared inside `CREATE TABLE` and can be modified with `ALTER TABLE`.

Column Definition Syntax

SYNTAX — COLUMN DEFINITION

```
COLUMN_name  data_type  [NOT NULL | NULL]
               [DEFAULT DEFAULT_value]
               [UNIQUE]
               [PRIMARY KEY]
               [CHECK (expression)]
               [AUTO_INCREMENT]
               [REFERENCES other_TABLE(other_col)]
               [COMMENT 'description']
```

Note: Multiple constraints can be combined on the same column

Common Data Types Reference

Category	Data Type	Storage	Use Case / Example
Integer	TINYINT	1 byte	Age, status flags (0–255)
Integer	SMALLINT	2 bytes	Year, small counters
Integer	INT / INTEGER	4 bytes	IDs, counts, quantities
Integer	BIGINT	8 bytes	Large IDs, Unix timestamps
Decimal	DECIMAL(p,s)	Variable	Money: DECIMAL(10,2) → 99999999.99
Float	FLOAT / DOUBLE	4/8 bytes	Scientific values, coordinates
String	CHAR(n)	n bytes fixed	Fixed codes: CHAR(2) for country code
String	VARCHAR(n)	Up to n bytes	Names, emails: VARCHAR(100)
String	TEXT	Up to 65KB	Articles, descriptions
String	LONGTEXT	Up to 4GB	Full documents, HTML content
Date/Time	DATE	3 bytes	2024-06-01 — birthdays, events
Date/Time	DATETIME	8 bytes	2024-06-01 14:30:00 — timestamps
Date/Time	TIMESTAMP	4 bytes	Auto-records row creation time
Boolean	BOOLEAN / BOOL	1 byte	TRUE/FALSE, active flags
Binary	BLOB	Up to 65KB	Small images, binary files

Category	Data Type	Storage	Use Case / Example
JSON	JSON	Variable	Semi-structured data (MySQL 5.7+)
Special	ENUM('a','b')	1-2 bytes	Fixed set: ENUM('Active','Inactive')

Adding & Modifying Columns — Syntax

SYNTAX — ALTER TABLE — Column Operations

```
-- ADD COLUMN at end
ALTER TABLE TABLE_name ADD COLUMN col_name datatype [CONSTRAINTS];

-- ADD COLUMN at specific position (MySQL)
ALTER TABLE TABLE_name ADD COLUMN col_name datatype AFTER other_col;

-- Change data type or CONSTRAINTs
ALTER TABLE TABLE_name MODIFY COLUMN col_name new_datatype [CONSTRAINTS];

-- Rename + change type (MySQL 8+ / PostgreSQL)
ALTER TABLE TABLE_name CHANGE old_name new_name datatype;      -- MySQL
ALTER TABLE TABLE_name RENAME COLUMN old_name TO new_name;      -- PostgreSQL

-- DROP a COLUMN
ALTER TABLE TABLE_name DROP COLUMN col_name;
```

```
ALTER TABLE Employees ADD COLUMN Phone VARCHAR(15) AFTER Email;
ALTER TABLE Employees ADD COLUMN IsActive BOOLEAN DEFAULT TRUE;
ALTER TABLE Employees MODIFY COLUMN Salary DECIMAL(12,2) NOT NULL;
ALTER TABLE Employees DROP COLUMN TempNotes;
```

4

Primary Key (PK)

Uniquely identifies every row — syntax for all PK types

A **Primary Key** enforces **UNIQUE + NOT NULL** on the key column(s). Every table must have exactly one PK. It can be defined inline or as a table constraint.

Primary Key — All Syntax Forms

SYNTAX — PRIMARY KEY

```
-- Form 1: Inline ON the COLUMN (single COLUMN PK)
CREATE TABLE T ( id INT PRIMARY KEY, ... );

-- Form 2: TABLE-level CONSTRAINT (single or composite)
CREATE TABLE T (
    col1 datatype,
    col2 datatype,
    PRIMARY KEY (col1)           -- single
);

-- Form 3: Composite PK (multiple COLUMNS)
CREATE TABLE Enrollments (
    StudentID INT,
    CourseID INT,
    PRIMARY KEY (StudentID, CourseID) -- both together must be UNIQUE
);

-- Form 4: ADD PK to existing TABLE
ALTER TABLE TABLE_name ADD PRIMARY KEY (col_name);

-- Form 5: DROP PK
ALTER TABLE TABLE_name DROP PRIMARY KEY;
```

Surrogate Key with AUTO_INCREMENT / SERIAL

SYNTAX — AUTO-GENERATED PRIMARY KEY

```
-- MySQL / MariaDB
CREATE TABLE Products (
    ProductID INT PRIMARY KEY AUTO_INCREMENT,
    Name VARCHAR(100) NOT NULL
);

-- PostgreSQL (SERIAL is shorthand for INT + sequence)
CREATE TABLE Products (
    ProductID SERIAL PRIMARY KEY,
    Name VARCHAR(100) NOT NULL
);

-- PostgreSQL modern syntax (GENERATED ALWAYS AS IDENTITY)
CREATE TABLE Products (
    ProductID INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    Name VARCHAR(100) NOT NULL
);

-- Get the last auto-generated ID after INSERT
SELECT LAST_INSERT_ID(); -- MySQL
SELECT LASTVAL();        -- PostgreSQL
```

Named PK Constraint

```
CREATE TABLE Orders ( OrderID INT NOT NULL, CustomerID INT NOT NULL, OrderDate DATE,
CONSTRAINT pk_orders PRIMARY KEY (OrderID) -- named constraint ); -- Dropping a named
constraint is easier: ALTER TABLE Orders DROP CONSTRAINT pk_orders;
```

A **Foreign Key** links a column in the child table to the **Primary Key** of a parent table, ensuring that only valid referenced values can be stored.

Foreign Key — All Syntax Forms

SYNTAX — FOREIGN KEY

```
-- Form 1: Inline REFERENCES (single COLUMN)
CREATE TABLE T (
  col datatype REFERENCES parent_TABLE(parent_col)
);

-- Form 2: TABLE-level FOREIGN KEY CONSTRAINT
CREATE TABLE T (
  col datatype,
  FOREIGN KEY (col) REFERENCES parent_TABLE(parent_col)
);

-- Form 3: Named FK WITH FULL CASCADE options
CREATE TABLE T (
  col datatype,
  CONSTRAINT fk_name
    FOREIGN KEY (col)
    REFERENCES parent_TABLE(parent_col)
    ON DELETE CASCADE
    ON UPDATE CASCADE
);

-- Form 4: ADD FK to existing TABLE
ALTER TABLE child_TABLE
ADD CONSTRAINT fk_name
FOREIGN KEY (col) REFERENCES parent_TABLE(parent_col)
ON DELETE RESTRICT ON UPDATE CASCADE;

-- Form 5: DROP FK
ALTER TABLE child_TABLE DROP FOREIGN KEY fk_name; -- MySQL
ALTER TABLE child_TABLE DROP CONSTRAINT fk_name; -- PostgreSQL
```

ON DELETE / ON UPDATE Options

Option	ON DELETE Behaviour	ON UPDATE Behaviour	Best Use Case
CASCADE	Child rows automatically deleted when parent row is deleted	Child rows automatically updated when parent row is updated	Enforces 1:1 relationship where both sides must stay in sync
SET NULL	FK column set to NULL in child row	FK column set to NULL	Orders when Customer deleted (keep order history)

Option	ON DELETE Behaviour	ON UPDATE Behaviour	Best Use Case
SET DEFAULT	FK set to column's DEFAULT value	FK set to column's DEFAULT value	Reassign to a default department
RESTRICT	Parent delete BLOCKED if child rows exist	Parent update BLOCKED if child rows exist	Prevent accidental deletion of parents
NO ACTION	Same as RESTRICT (checked at end of statement)	Same as RESTRICT	Standard SQL default behaviour

Practical FK Examples:

```
-- Students belong to a Department
CREATE TABLE Students (
  StudentID INT PRIMARY KEY
  AUTO_INCREMENT,
  Name VARCHAR(100) NOT NULL,
  DeptID INT,
  CONSTRAINT fk_student_dept
  FOREIGN KEY (DeptID) REFERENCES Departments(DeptID)
  ON DELETE SET NULL -- student stays if dept is deleted
  ON UPDATE CASCADE -- auto-update if DeptID changes
);
-- Enrollments cascade-delete when student or course is deleted
CONSTRAINT fk_enroll_student FOREIGN KEY (StudentID)
REFERENCES Students(StudentID) ON DELETE CASCADE,
CONSTRAINT fk_enroll_course FOREIGN KEY (CourseID)
REFERENCES Courses(CourseID) ON DELETE CASCADE
```

A **schema** (or **database** in MySQL) is the logical container that groups all related tables, views, indexes, and other objects. You must create and select a schema before creating tables.

Database / Schema Management Syntax

SYNTAX — DATABASE / SCHEMA

```
-- CREATE a DATABASE (MySQL / MariaDB)
CREATE DATABASE [IF NOT EXISTS] db_name
    [CHARACTER SET charSET_name]
    [COLLATE collation_name];

-- CREATE a schema (PostgreSQL / SQL standard)
CREATE SCHEMA [IF NOT EXISTS] schema_name [AUTHORIZATION owner];

-- SELECT / switch to a DATABASE
USE db_name;                -- MySQL
SET search_path TO schema_name; -- PostgreSQL

-- List all DATABASEs
SHOW DATABASES;             -- MySQL
SELECT datname FROM pg_DATABASE; -- PostgreSQL

-- DROP a DATABASE
DROP DATABASE [IF EXISTS] db_name;

-- Show current DATABASE
SELECT DATABASE();           -- MySQL
SELECT current_DATABASE();    -- PostgreSQL
```

SHOW & DESCRIBE — Inspect Schema

SYNTAX — INSPECT SCHEMA

```
-- List all TABLEs in current DATABASE
SHOW TABLES;                                -- MySQL
SELECT TABLE_name FROM information_schema.TABLES WHERE TABLE_schema = 'db';

-- Show structure (COLUMNs) of a TABLE
DESCRIBE TABLE_name;                        -- MySQL shorthand
DESC TABLE_name;                            -- same
SHOW COLUMNS FROM TABLE_name;             -- MySQL verbose

-- Show the CREATE TABLE statement used
SHOW CREATE TABLE TABLE_name;

-- List INDEXes ON a TABLE
SHOW INDEX FROM TABLE_name;                 -- MySQL
SELECT * FROM information_schema.INDEXes;     -- standard SQL

-- Show all CONSTRAINTs
SELECT * FROM information_schema.TABLE_CONSTRAINTS WHERE TABLE_name = 'T';
```

Complete Schema Example — College Database

```
CREATE DATABASE IF NOT EXISTS CollegeDB CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci; USE CollegeDB; CREATE TABLE Departments ( DeptID INT PRIMARY KEY
AUTO_INCREMENT, DeptName VARCHAR(80) NOT NULL UNIQUE, HoD VARCHAR(100) ); CREATE TABLE
Students ( StudentID INT PRIMARY KEY AUTO_INCREMENT, FullName VARCHAR(100) NOT NULL,
Email VARCHAR(100) UNIQUE NOT NULL, Age INT CHECK(Age BETWEEN 16 AND 60), DeptID INT,
CONSTRAINT fk_dept FOREIGN KEY (DeptID) REFERENCES Departments(DeptID) ON DELETE SET
NULL ); CREATE TABLE Courses ( CourseID INT PRIMARY KEY AUTO_INCREMENT, Title
VARCHAR(100) NOT NULL, Credits TINYINT CHECK(Credits BETWEEN 1 AND 6) ); CREATE TABLE
Enrollments ( StudentID INT NOT NULL, CourseID INT NOT NULL, Grade CHAR(2), PRIMARY KEY
(StudentID, CourseID), FOREIGN KEY (StudentID) REFERENCES Students(StudentID) ON DELETE
CASCADE, FOREIGN KEY (CourseID) REFERENCES Courses(CourseID) ON DELETE CASCADE );
```

An **index** is a sorted auxiliary structure that allows the database to find rows without scanning the entire table. It dramatically speeds up **SELECT WHERE**, **JOIN ON**, and **ORDER BY** operations.

CREATE INDEX — Full Syntax

SYNTAX — CREATE INDEX

```
-- Standard INDEX
CREATE INDEX INDEX_name ON TABLE_name (COLUMN1 [ASC|DESC]);

-- UNIQUE INDEX (also enforces no duplicates)
CREATE UNIQUE INDEX INDEX_name ON TABLE_name (COLUMN1);

-- Composite INDEX (multi-COLUMN)
CREATE INDEX INDEX_name ON TABLE_name (col1, col2, col3);

-- FULL-text INDEX (MySQL)
CREATE FULLTEXT INDEX INDEX_name ON TABLE_name (col1, col2);

-- INDEX ONLY if it doesn't exist (MySQL 8+)
CREATE INDEX IF NOT EXISTS INDEX_name ON TABLE_name (col);

-- Partial INDEX — INDEX ONLY matching rows (PostgreSQL)
CREATE INDEX INDEX_name ON TABLE_name (col) WHERE status = 'active';

-- Covering INDEX — include extra COLUMNS (PostgreSQL)
CREATE INDEX INDEX_name ON TABLE_name (col1) INCLUDE (col2, col3);

-- DROP an INDEX
DROP INDEX INDEX_name ON TABLE_name;    -- MySQL
DROP INDEX INDEX_name;                    -- PostgreSQL
```

Practical Index Examples

```
-- Speed up login: finding student by email CREATE UNIQUE INDEX idx_student_email ON
Students(Email); -- Speed up listing all students in a department CREATE INDEX
idx_student_dept ON Students(DeptID); -- Speed up searching courses by title (full-text)
CREATE FULLTEXT INDEX idx_course_title ON Courses(Title); -- Composite: queries
filtering by DeptID then sorted by LastName CREATE INDEX idx_dept_name ON
Students(DeptID, LastName); -- Check if index is being used (EXPLAIN) EXPLAIN SELECT *
FROM Students WHERE Email = 'arjun@college.edu'; -- Look for 'key: idx_student_email' in
the output
```

EXPLAIN — Analyse Query Performance

SYNTAX — EXPLAIN

```
-- Show query execution plan (did it use an INDEX?)
EXPLAIN SELECT * FROM TABLE_name WHERE col = value;

-- PostgreSQL detailed plan
EXPLAIN ANALYZE SELECT * FROM TABLE_name WHERE col = value;

-- Key COLUMNS to CHECK in EXPLAIN output:
--   type: 'ref' or 'range' = INDEX used (GOOD)
--   type: 'ALL'           = FULL TABLE scan (BAD - ADD INDEX)
--   key:  shows which INDEX WAS used
--   rows: estimated rows examined
```

Index Type	How It Works	When to Use	Avoid When
B-Tree (default)	Sorted tree; O(log n) lookup	Equality, range, ORDER BY	Very low cardinality columns
Hash	Hash function; O(1) equality	Equality only (=, IN)	Range queries (<, >, BETWEEN)
Full-Text	Inverted word index	LIKE '%word%', natural language	Exact value lookups
Composite	B-Tree on multiple cols	Multi-column WHERE clauses	Wrong column order
Partial	Index subset of rows	Filtered queries (WHERE stable)	Where filter is rarely used
Clustered	Rows stored in PK order	PK lookups, range on PK	Frequent PK changes

Constraints are **rules enforced by the database engine** to guarantee data integrity. Any INSERT, UPDATE, or DELETE that violates a constraint is automatically rejected.

NOT NULL — Syntax & Examples

SYNTAX — NOT NULL

```
-- ON CREATE TABLE (inline)
col_name datatype NOT NULL;

-- ADD NOT NULL to existing COLUMN (ALTER)
ALTER TABLE TABLE_name MODIFY COLUMN col_name datatype NOT NULL;

-- Remove NOT NULL (allow NULLs again)
ALTER TABLE TABLE_name MODIFY COLUMN col_name datatype NULL;
```

Note: Every PRIMARY KEY column is automatically NOT NULL

```
-- Correct: NULL not allowed in Email or FullName CREATE TABLE Students ( StudentID INT
PRIMARY KEY AUTO_INCREMENT, FullName VARCHAR(100) NOT NULL, Email VARCHAR(100) NOT NULL,
Phone VARCHAR(15) -- Phone CAN be NULL (optional) ); -- This INSERT will FAIL: INSERT
INTO Students (FullName, Email) VALUES (NULL, 'test@email.com'); -- ERROR: Column
'FullName' cannot be null
```

UNIQUE — Syntax & Examples

SYNTAX — UNIQUE

```
-- Inline single-COLUMN UNIQUE
col_name datatype UNIQUE;

-- Named UNIQUE CONSTRAINT (TABLE-level)
CONSTRAINT uq_name UNIQUE (col1);

-- Composite UNIQUE (combination must be UNIQUE)
CONSTRAINT uq_name UNIQUE (col1, col2);

-- ADD UNIQUE to existing TABLE
ALTER TABLE TABLE_name ADD UNIQUE (col_name);
ALTER TABLE TABLE_name ADD CONSTRAINT uq_name UNIQUE (col1, col2);

-- DROP UNIQUE CONSTRAINT
ALTER TABLE TABLE_name DROP INDEX uq_name;           -- MySQL
ALTER TABLE TABLE_name DROP CONSTRAINT uq_name;       -- PostgreSQL
```

Note: UNIQUE allows one NULL unless combined with NOT NULL

```
CREATE TABLE Employees ( EmpID INT PRIMARY KEY AUTO_INCREMENT, Email VARCHAR(100) UNIQUE
NOT NULL, -- no duplicate emails Phone VARCHAR(15) UNIQUE, -- unique but can be NULL
FirstName VARCHAR(50) NOT NULL, LastName VARCHAR(50) NOT NULL, CONSTRAINT uq_fullname
UNIQUE (FirstName, LastName) -- composite unique ); -- This will FAIL if email already
exists: INSERT INTO Employees (Email) VALUES ('existing@email.com'); -- ERROR: Duplicate
entry 'existing@email.com' for key 'Email'
```

CHECK — Syntax & Examples

SYNTAX — CHECK

```
-- Inline CHECK
col_name datatype CHECK (expression);

-- Named CHECK CONSTRAINT (recommended)
CONSTRAINT chk_name CHECK (expression);

-- TABLE-level CHECK (can reference multiple COLUMNS)
CONSTRAINT chk_date_range CHECK (EndDate > StartDate);

-- ADD CHECK to existing TABLE
ALTER TABLE TABLE_name ADD CONSTRAINT chk_name CHECK (expression);

-- DROP CHECK CONSTRAINT
ALTER TABLE TABLE_name DROP CONSTRAINT chk_name;       -- PostgreSQL
ALTER TABLE TABLE_name DROP CHECK chk_name;             -- MySQL 8+
```

Note: CHECK expressions can reference any column in the same row

```
CREATE TABLE Employees ( EmpID INT PRIMARY KEY AUTO_INCREMENT, Age INT CHECK(Age BETWEEN 18 AND 65), Salary DECIMAL(10,2) CHECK(Salary >= 15000), Gender CHAR(1) CHECK(Gender IN ('M','F','O')), HireDate DATE NOT NULL, EndDate DATE, CONSTRAINT chk_dates CHECK(EndDate IS NULL OR EndDate > HireDate), CONSTRAINT chk_salary CHECK(Salary <= 10000000) -- max 1 crore ); -- This will FAIL: INSERT INTO Employees (Age, Salary) VALUES (15, 5000); -- ERROR: CHECK constraint 'Age' is violated
```

DEFAULT — Syntax & Examples

SYNTAX — DEFAULT

```
-- Static DEFAULT value
col_name datatype DEFAULT literal_value;

-- Dynamic DEFAULT (function)
CREATEDAt DATETIME DEFAULT CURRENT_TIMESTAMP;
UPDATEDAt DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP;

-- ADD DEFAULT to existing COLUMN
ALTER TABLE TABLE_name ALTER COLUMN col_name SET DEFAULT value; -- PostgreSQL
ALTER TABLE TABLE_name MODIFY COLUMN col_name datatype DEFAULT v; -- MySQL

-- Remove DEFAULT
ALTER TABLE TABLE_name ALTER COLUMN col_name DROP DEFAULT;

-- Override DEFAULT explicitly in INSERT
INSERT INTO T (col) VALUES (DEFAULT); -- use the defined DEFAULT
```

```
CREATE TABLE Orders ( OrderID INT PRIMARY KEY AUTO_INCREMENT, CustomerID INT NOT NULL, Status VARCHAR(20) DEFAULT 'Pending', -- text default Quantity INT DEFAULT 1, -- numeric default Discount DECIMAL(5,2) DEFAULT 0.00, -- zero discount OrderDate DATE DEFAULT (CURRENT_DATE), -- today's date CreatedAt DATETIME DEFAULT CURRENT_TIMESTAMP -- auto timestamp ); -- Only CustomerID needed – rest use defaults: INSERT INTO Orders (CustomerID) VALUES (42); -- Result: Status='Pending', Quantity=1, Discount=0.00, OrderDate=today
```

Relationships are defined through Foreign Keys and queried using **JOIN** statements. Understanding join types is essential for retrieving data across multiple tables.

JOIN — Complete Syntax Reference

SYNTAX — JOIN SYNTAX

```
SELECT t1.col, t2.col
FROM   TABLE1 t1
[INNER | LEFT | RIGHT | FULL OUTER | CROSS] JOIN TABLE2 t2
      ON t1.key = t2.foreign_key
[WHERE condition]
[GROUP BY col] [HAVING condition]
[ORDER BY col [ASC|DESC]]
[LIMIT n];
```

JOIN Type	Returns	Use When	Syntax Keyword
INNER JOIN	Only rows that have a match in BOTH tables	You only want records with a valid relationship	INNER JOIN
LEFT JOIN (LEFT OUTER JOIN)	Rows from left table + matching rows from right table (rows from left table even if no match)	Get all from left table (rows from right table only if they match)	LEFT JOIN
RIGHT JOIN (RIGHT OUTER JOIN)	Rows from right table + matching rows from left table (rows from right table even if no match)	Get all from right table (rows from left table only if they match)	RIGHT JOIN
FULL OUTER JOIN	ALL rows from both tables (NULL where matched rows are missing)	Find unmatched rows in either table	FULL OUTER JOIN (PostgreSQL; simulate in MySQL)
CROSS JOIN	Cartesian product — every row of one table with every row of another	Generate all combinations (e.g. Cross-Joins)	CROSS JOIN
SELF JOIN	Join a table with itself	Hierarchical data (employee → manager in same table)	JOIN table AS alias

JOIN Examples for All Relationship Types

```
-- ■■ 1:N ■■ One Department has many Students -- All students with their department
name (INNER JOIN) SELECT s.FullName, d.DeptName FROM Students s INNER JOIN Departments d
ON s.DeptID = d.DeptID; -- All departments even if they have no students (LEFT JOIN)
SELECT d.DeptName, s.FullName FROM Departments d LEFT JOIN Students s ON s.DeptID =
d.DeptID; -- ■■ M:N ■■ Students enrolled in Courses (via Enrollments) -- All courses a
specific student is enrolled in SELECT s.FullName, c.Title, e.Grade FROM Students s
INNER JOIN Enrollments e ON e.StudentID = s.StudentID INNER JOIN Courses c ON c.CourseID
= e.CourseID WHERE s.StudentID = 1; -- ■■ 1:1 ■■ Employee and their Profile SELECT
e.FullName, p.BloodGroup, p.EmergencyContact FROM Employees e LEFT JOIN EmployeeProfiles
p ON p.EmpID = e.EmpID; -- ■■ SELF JOIN ■■ Employee with their Manager (same table)
SELECT e.FullName AS Employee, m.FullName AS Manager FROM Employees e LEFT JOIN
Employees m ON e.ManagerID = m.EmpID;
```

Aggregate Queries with GROUP BY

SYNTAX — GROUP BY / HAVING

```
SELECT    col, AGG_FUNCTION(col2)
FROM      TABLE_name
[JOIN     ...]
WHERE     row_filter_cONdition      -- filter BEFORE grouping
GROUP BY  col
HAVING    group_filter_cONdition    -- filter AFTER grouping
ORDER BY  col [ASC|DESC]
LIMIT     n;

-- Aggregate functiONs: COUNT, SUM, AVG, MIN, MAX
```

```
-- Count of students per department SELECT d.DeptName, COUNT(s.StudentID) AS
StudentCount FROM Departments d LEFT JOIN Students s ON s.DeptID = d.DeptID GROUP BY
d.DeptName HAVING COUNT(s.StudentID) > 2 -- only depts with more than 2 students ORDER
BY StudentCount DESC; -- Average grade per course SELECT c.Title, AVG(e.Marks) AS
AvgMarks, MAX(e.Marks) AS TopScore FROM Courses c JOIN Enrollments e ON e.CourseID =
c.CourseID GROUP BY c.CourseID, c.Title;
```

A **transaction** is a group of SQL statements that execute as one atomic unit. ACID properties guarantee correctness and reliability for every transaction.

Transaction Control Syntax

SYNTAX — TRANSACTION CONTROL

```
-- Start a TRANSACTION
BEGIN;                                -- PostgreSQL / SQLite
START TRANSACTION;                   -- MySQL
BEGIN TRANSACTION;                   -- SQL Server

-- Permanently save all changes
COMMIT;

-- Undo ALL changes since BEGIN
ROLLBACK;

-- CREATE a named ROLLBACK point inside a TRANSACTION
SAVEPOINT SAVEPOINT_name;

-- ROLLBACK TO a specific SAVEPOINT (partial undo)
ROLLBACK TO SAVEPOINT SAVEPOINT_name;

-- Remove a SAVEPOINT
RELEASE SAVEPOINT SAVEPOINT_name;

-- SET isolation level for next TRANSACTION
SET TRANSACTION ISOLATION LEVEL [level];
-- Levels: READ UNCOMMITTED | READ COMMITTED | REPEATABLE READ | SERIALIZABLE
```

Full Transaction Examples

```
-- ■■ Atomicity Example: Bank Transfer ■■ START TRANSACTION; UPDATE Accounts SET
Balance = Balance - 5000 WHERE AccID = 'A001'; UPDATE Accounts SET Balance = Balance +
5000 WHERE AccID = 'B002'; -- Check if both went OK before committing -- (Application
logic: if error → ROLLBACK, else → COMMIT) COMMIT; -- Both updates saved permanently --
OR: ROLLBACK; -- If any step failed – BOTH updates undone -- ■■ SAVEPOINT Example:
Multi-step Order Processing ■■ START TRANSACTION; INSERT INTO Orders (CustomerID, Total)
VALUES (101, 2499.00); SAVEPOINT order_created; -- safe point after order insert INSERT
INTO OrderItems (OrderID, ProductID, Qty) VALUES (LAST_INSERT_ID(), 5, 2); SAVEPOINT
items_added; -- safe point after items UPDATE Inventory SET Stock = Stock - 2 WHERE
ProductID = 5; -- If inventory update fails: ROLLBACK TO SAVEPOINT items_added; -- undo
inventory, keep order+items COMMIT;
```

Isolation Levels — Syntax & Comparison

SYNTAX — SET TRANSACTION ISOLATION LEVEL

```
-- SET for current session
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;  -- MySQL
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;             -- SQL standard

-- PostgreSQL
BEGIN ISOLATION LEVEL REPEATABLE READ;

-- SET globally (MySQL)
SET GLOBAL TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read	Performance	Use Case
READ UNCOMMITTED	Possible	Possible	Possible	Fastest	Logging, analytics (rarely used)
READ COMMITTED	Prevented	Possible	Possible	Fast	Most web apps — PostgreSQL default
REPEATABLE READ	Prevented	Prevented	Possible	Moderate	MySQL default; financial reads
SERIALIZABLE	Prevented	Prevented	Prevented	Slowest	Banking, stock trading, auditing

ACID Quick-Reference with SQL Keywords

Property	Guarantee	SQL Keywords	Example Scenario
Atomicity (A)	All or nothing — partial updates never happen	BEGIN, COMMIT, ROLLBACK	Bank transfer: debit + credit both succeed or both fail
Consistency (C)	DB moves from one valid state to another	CONSTRAINT, CHECK, FK, NOT NULL	BALANCE CHECK >= 0 prevents negative account balance
Isolation (I)	Concurrent transactions don't interfere	SET TRANSACTION ISOLATION LEVEL	Two users booking last seat — only one succeeds
Durability (D)	Committed data survives crashes	COMMIT (writes to WAL / redo log)	Confirmed payment persists even if server crashes

■ Key Takeaway

Always wrap multi-step data modifications in a transaction. Use SAVEPOINT for complex workflows where partial rollback may be needed. Choose your isolation level carefully — SERIALIZABLE is safest but slowest; READ COMMITTED is the